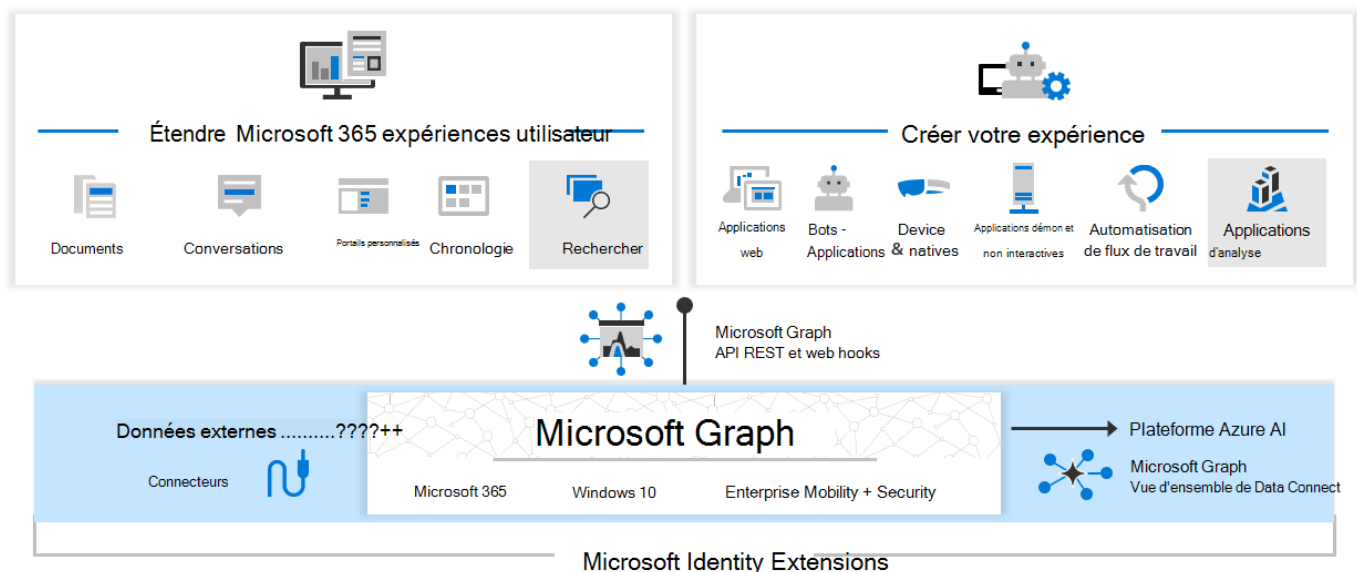


Présentation de Microsoft Graph

Introduction (source : [Microsoft Learn](#))

Microsoft Graph est une passerelle qui vous permet d'accéder aux données et aux renseignements dans Microsoft 365. Elle fournit un modèle de programmabilité unifié qui vous permet d'accéder à la quantité impressionnante de données disponibles dans Microsoft 365, Windows et Enterprise Mobility + Security. Utiliser la richesse des données accessibles via Microsoft Graph pour créer des applications pour les organisations et les consommateurs qui interagissent avec des millions d'utilisateurs

Microsoft 365 Plateforme



Il existe un point de terminaison unique à Microsoft Graph : [Microsoft Graph](#), permettant de regrouper toutes ses actions dans l'environnement Microsoft

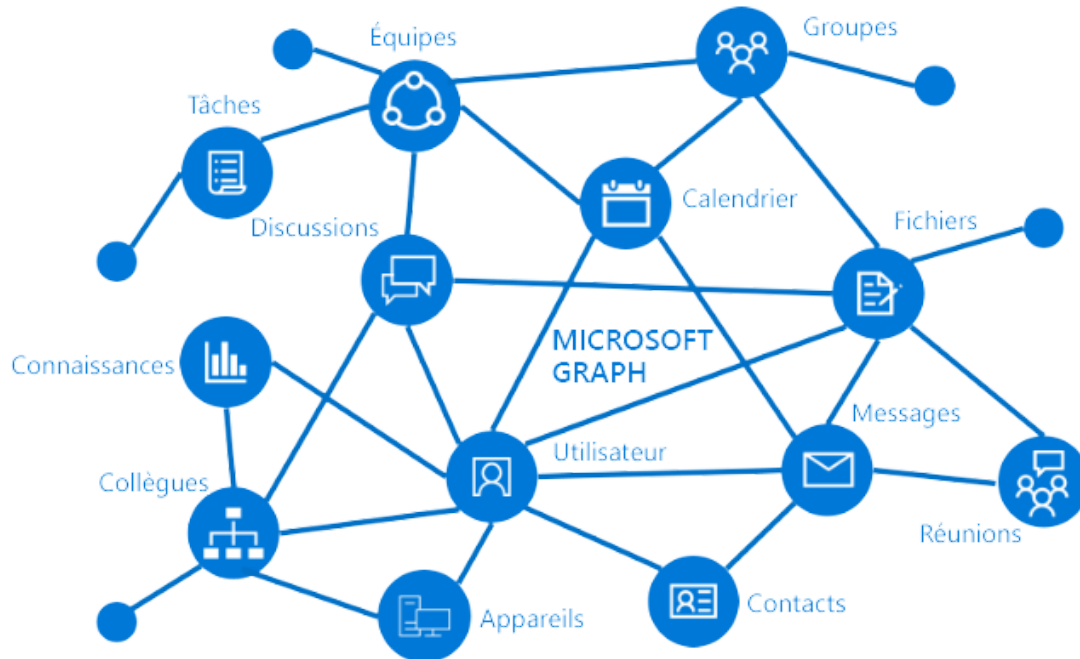
"Ensemble, l'API Microsoft Graph, les connecteurs et Connexion aux données alimentent la plateforme Microsoft 365. Dans la mesure où vous avez la possibilité d'accéder aux données Microsoft Graph et à d'autres jeux de données, vous pouvez en extraire des insights et des analyses, vous pouvez étendre les expériences Microsoft 365 et créer des applications uniques et intelligentes."

Nouveautés de Microsoft Graph

Microsoft Graph propose des **API REST** et des bibliothèques clientes pour accéder à des données sur les services cloud Microsoft suivants :

Services principaux Microsoft 365 : Bookings, Calendrier, Delve, Excel, Microsoft 365 compliance eDiscovery, Microsoft Search, OneDrive, OneNote, Outlook/Exchange, Personnes (contacts Outlook), Planificateur, SharePoint, Teams, To Do, Viva Insights Enterprise Mobility + Security services : Advanced Threat Analytics, Advanced Threat Protection, Microsoft Entra ID, Identity Manager et Intune Services Windows : activités, appareils, notifications, Impression universelle Services Dynamics 365 Business Central

Pour en savoir plus, consultez [Principaux services et fonctionnalités dans Microsoft Graph](#).



Installation de Microsoft Graph SDK

Pré-requis

PowerShell 7 est conseillé pour utiliser Graph sur toutes les plateformes. Il n'y a pas d'autre prérequis à l'installation de **Graph SDK** sous cette version de PowerShell

Sous **Windows PowerShell**, il y a des prérequis :

- Avoir Windows PowerShell 5.1 minimum
- Installer .NET Framework 4.7.2 minimum
- Mettre à jour PowerShellGet vers la dernière version en utilisant :

```
Install-Module PowerShellGet
```

- La restriction d'exécution des scripts PowerShell doit être à **remote signed** ou moins restrictif en utilisant :

```
Set-ExecutionPolicy -ExecutionPolicy RemoteSigned -Scope CurrentUser
```

Le SDK Microsoft Graph PowerShell se présente sous la forme de **deux modules**, Microsoft.Graph et Microsoft.Graph.Beta, que vous installerez séparément. Ces modules appellent respectivement les points de terminaison Microsoft Graph v1.0 et Microsoft Graph beta. Vous pouvez installer les deux modules sur la même version de PowerShell.

[!CAUTION] L'installation des modules principaux du SDK, Microsoft.Graph et Microsoft.Graph.Beta, entraînera l'installation des 38 sous-modules de chaque module. Pensez à n'installer que les modules nécessaires, y compris **Microsoft.Graph.Authentication** qui est installé par défaut lorsque vous choisissez d'installer les sous-modules individuellement. Pour obtenir une liste des modules Microsoft

Graph disponibles, utilisez `Find-Module Microsoft.Graph*`. Seules les cmdlets des modules installés seront disponibles.

Pour installer la v1 du module **Microsoft.Graph**, sur Windows PowerShell ou PowerShell Core, lancer la commande suivante.

```
Install-Module Microsoft.Graph -Scope CurrentUser
```

En option, vous pouvez modifier l'**étendue de l'installation** à l'aide du paramètre `-Scope`. Cela nécessite des droits d'administrateur.

```
Install-Module Microsoft.Graph -Scope AllUsers
```

Pour installer le module en beta, utilisez la commande suivante.

```
Install-Module Microsoft.Graph.Beta
```

[!NOTE] Nous vous recommandons de toujours utiliser **Microsoft Graph v1.0** lorsque vous écrivez des scripts. Il est parfois nécessaire d'utiliser le point de terminaison bêta à des fins de test ou d'adoption anticipée avant qu'une fonctionnalité ne soit disponible dans la version 1.0. Le point de terminaison bêta de Microsoft Graph et toute fonctionnalité qui s'y trouve sont encore à **l'état d'avant-première** et peuvent changer. Les points de terminaison bêta ne sont donc pas fiables pour une utilisation en production, car ils peuvent interrompre des scénarios existants sans préavis.

L'installation du SDK dans une version de PowerShell ne l'installe pas dans l'autre. **Exécutez la commande d'installation dans la version de PowerShell dans laquelle vous avez l'intention de l'utiliser.**

Vérification de l'installation

Lorsque l'installation est finie, il est possible de vérifier la version installée avec la commande suivante.

```
Get-InstalledModule Microsoft.Graph
```

Pour vérifier les sous-modules installés et leurs versions, lancer la commande suivante :

```
Get-InstalledModule
```

La version dans le résultat de la commande doit correspondre à la dernière version publiée sur la [PowerShell Gallery](#).

Mettre à jour le SDK

Il est possible de mettre à jour le SDK et ses dépendances en utilisant la commande suivante.

```
Update-Module Microsoft.Graph
```

Désinstaller le SDK

Premièrement, utiliser la commande suivante pour désinstaller le module principal

```
Uninstall-Module Microsoft.Graph -AllVersions
```

Puis, désinstaller toutes ses dépendances en utilisant les commandes suivantes.

```
Get-InstalledModule Microsoft.Graph.* | ? Name -ne  
"Microsoft.Graph.Authentication" | Uninstall-Module  
Uninstall-Module Microsoft.Graph.Authentication
```

Se connecter à Microsoft Graph via PowerShell (source : [IT-Connect](#))

Lors de l'utilisation du module Microsoft Graph et du cmdlet Connect-MgGraph pour établir une connexion à votre tenant, vous avez le choix entre deux modes de connexion :

Une connexion avec un accès délégué

Avec le mode "**accès délégué**", il faut préciser les permissions dont vous avez besoin au moment de la connexion au tenant, et c'est en se connectant avec un compte qui a les droits que vous allez pouvoir créer cette délégation et permettre l'accès aux ressources.

Une connexion avec un accès application

Avec le mode "**accès application**", il est nécessaire d'**inscrire une nouvelle application** dans Entra ID afin de créer un **point de connexion**. Ensuite, il faudra accorder des autorisations à cette application pour que les scripts qui l'utilisent soient en mesure d'effectuer les actions déclarées dans votre code PowerShell. Pour se connecter via cette méthode, il faudra spécifier plusieurs informations : **ID du tenant**, **ID de l'application** et un **nom de certificat** ou **une empreinte de certificat** (qu'il faudra générer en amont).

PowerShell : Microsoft Graph via l'accès délégué

Commençons par étudier le principe de connexion via **l'accès délégué**, à partir de la commande **Connect-MgGraph**. Je vous rappelle que cette commande sert à établir une connexion à Microsoft Graph via PowerShell.

Lors de la connexion à Microsoft Graph, il faut spécifier **un ou plusieurs scopes (étendues)** en fonction de ce que vous cherchez à effectuer. Afin de pouvoir utiliser les étendues sélectionnées, une délégation d'accès sera créée au moment de la connexion.

Par exemple, si l'on souhaite **récupérer la liste des utilisateurs de notre tenant**, on a besoin d'un **accès en lecture aux utilisateurs**. Cela se traduit par le nom d'étendue suivant :

`User.Read.All`

Dans le cas où l'on aurait besoin de **créer un nouvel utilisateur**, l'étendue serait différente puisqu'il faudrait des **droits d'écriture**. L'étendue à utiliser serait :

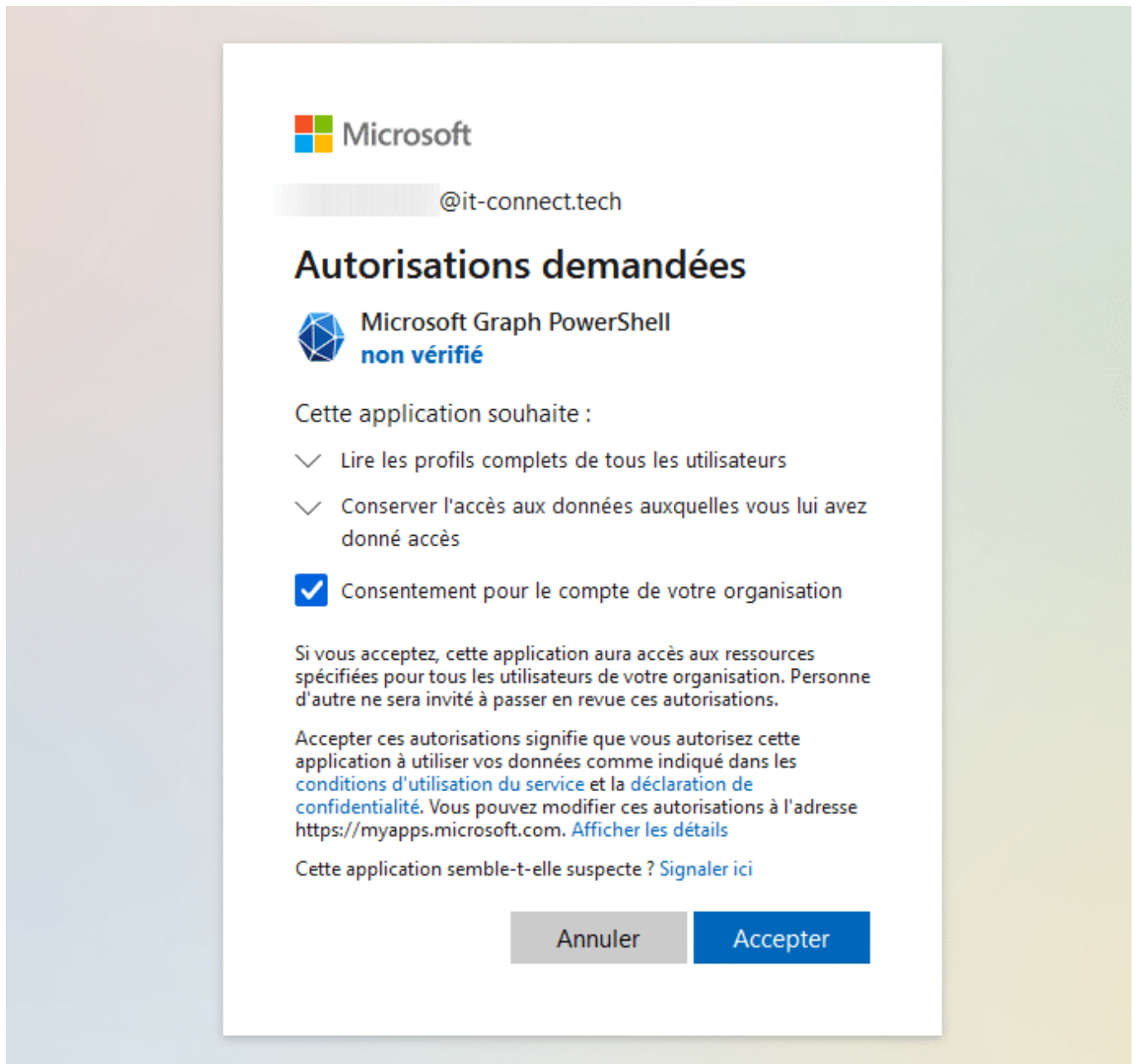
`User.ReadWrite.All`

Je vais vous donner quelques astuces par la suite pour identifier les étendues. Il faut savoir que l'on peut sélectionner plusieurs étendues, et que ce n'est pas anormal d'en sélectionner entre 5 et 10.

Continuons sur l'idée suivante : **récupérer la liste des utilisateurs, à partir de l'étendue "User.Read.All"**. La commande Connect-MgGraph à exécuter sera :

```
Connect-MgGraph -Scopes "User.Read.All"
```

Cette commande va **ouvrir un navigateur sur votre machine** afin d'effectuer la connexion et d'accorder l'accès à l'application "Microsoft Graph PowerShell". Cela nécessite un accès avec des droits administrateur. Il faudra **accepter la requête**, comme sur l'exemple ci-dessous.



Ensuite, un message s'affiche pour indiquer que la fenêtre peut être fermée : Authentication complete. You can return to the application. Feel free to close this browser tab.

Retour dans la console PowerShell. Un message **"Welcome To Microsoft Graph!"** doit être visible !

Pour **lister les utilisateurs**, on va tout simplement utiliser cette commande :

Get-MgUser

```
PS C:\Users\Administrateur> Get-MgUser
```

DisplayName	Id	Mail
UserPrincipalName		
-----	--	----

Alonso Balmasque	d04b0890-aef3-496e-bb26-c193b3056917	a.Balmasque@h122-16.lab-eni.fr
Agnes Moitrakil	e9dfa9a3-6496-4759-a48d-fae80e1e4d72	a.Moitrakil@h122-16.lab-

```

eni.fr a.Moitrankil@h122-16.lab-eni.fr
Globabl Admin 582ff086-8685-477c-aa72-460d48838ef0 admin@h122-16.lab-eni.fr
admin@h122-16.lab-eni.fr
MOD Administrator f22d7cdd-caef-4a0f-88e9-967570615160
admin@WWLx172224.onmicrosoft.com admin@WWLx172224.onmicrosoft.com
Brian May 16e6ba44-d234-4226-83df-d35c371660dd
B.May@h122-16.lab-eni.fr
Daisy Drate f22e4978-05dc-4679-b710-ac0078e709f3 d.Drate@h122-16.lab-
eni.fr d.Drate@h122-16.lab-eni.fr
Freddie Mercury 38158011-a6e5-4494-91d6-3d326fd9d171
f.mercury@h122-16.lab-eni.fr
Jean-Pierre Adressa 9c6e30b1-7821-41d9-b63d-91c7be5865a4 j.Adressa@h122-16.lab-
eni.fr j.Adressa@h122-16.lab-eni.fr
John Deacon 111d5103-5dfe-47fd-bf88-a097850bfa18
J.Deacon@h122-16.lab-eni.fr
Jean Tanrien 753ebc09-40b3-4904-ad6f-c501648d0184 j.Tanrien@h122-16.lab-
eni.fr j.Tanrien@h122-16.lab-eni.fr
labeni 60dfdb04-35c9-403f-b74c-50a740bec38c
labeni@h122-16.lab-eni.fr
Oussama LAIRBON 89c9951b-f901-40b1-a465-7df43447da26 oussama@h122-16.lab-
eni.fr oussama@h122-16.lab-eni.fr
Roger Taylor 136c574f-fec1-4870-8a87-4bf5d3242286
R.taylor@h122-16.lab-eni.fr
Sacha Touille c350720b-1738-4cab-b74a-4f77a7767a7a
sacha@h122-16.lab-eni.fr
Xavier Kafairgaf 92702c46-cb0b-4ed7-a199-d0adc72dcf91 x.Kafairgaf@h122-16.lab-
eni.fr x.Kafairgaf@h122-16.lab-eni.fr
Yves Remord 03a12f39-1d05-48cc-ab84-c4c4ce897bd9 y.Remord@h122-16.lab-
eni.fr y.Remord@h122-16.lab-eni.fr
Zachary Ramablag b532204a-506c-4832-bfc9-da9450de3c8e zachary@h122-16.lab-
eni.fr zachary@h122-16.lab-eni.fr

```

Si on essaye de lister les groupes présents, une erreur est retournée :

Get-MgGroup

```

Get-MgGroup : Insufficient privileges to complete the operation.
Status: 403 (Forbidden)
ErrorCode: Authorization_RequestDenied
Date: 2024-02-27T13:41:33
...

```

En fait, nous n'avons **pas le droit** de lister les groupes puisqu'on n'a pas précisé l'étendue qui permet de lire les groupes au moment de la connexion.

Dans la console PowerShell en cours, on peut exécuter une seconde fois **Connect-MgGraph** pour ajouter l'étendue **"Group.Read.All"** qui permettra de lire les informations des groupes.

```
Connect-MgGraph -Scopes "Group.Read.All"
```

Cela va venir **s'ajouter à la session déjà ouverte** et donc **conserver l'étendue "User.Read.All"**. Suite à l'exécution de cette commande, il faut de nouveau **approuver la connexion via le navigateur**. Désormais, on peut récupérer la liste des groupes.

La prochaine fois, **vous pouvez appeler directement les deux étendues** :

```
Connect-MgGraph -Scopes "User.Read.All","Group.Read.All"
```

Du fait que cette méthode nécessite une interaction et une validation via le navigateur, **elle n'est pas faite pour être utilisée dans des scripts**. La seconde méthode que nous allons voir dès maintenant sera plus adaptée.

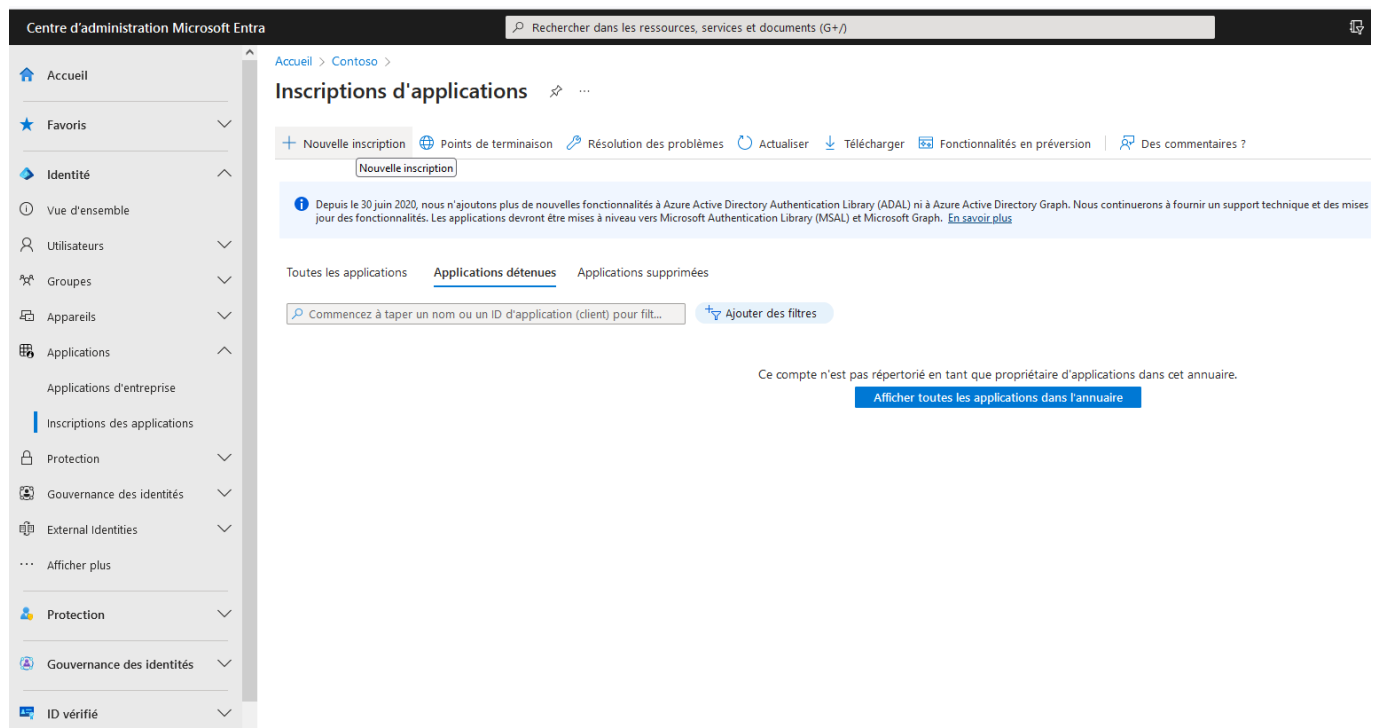
Juste avant cela, on va **se déconnecter proprement** de Microsoft Graph :

```
Disconnect-MgGraph
```

PowerShell : Microsoft Graph via l'accès application

Nous devons **inscrire une nouvelle application au sein d'Entra ID**, puis ensuite lui **accorder des autorisations**. Cette application sera notre point d'entrée avec PowerShell.

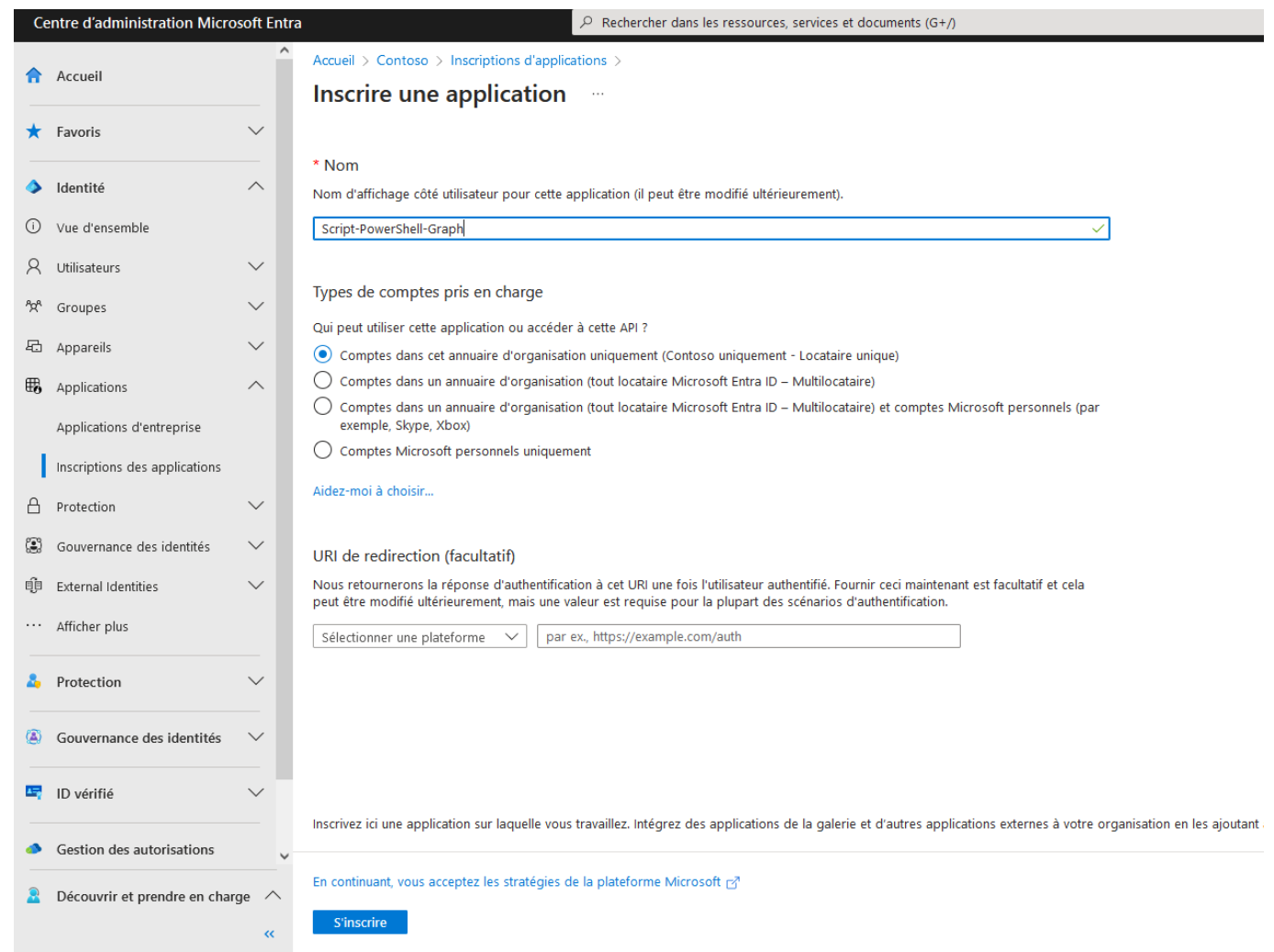
À partir du portail Identity, dans Entra ID, cliquez sur **"Inscriptions d'applications"** à gauche puis sur le bouton **"Nouvelle inscription"**.



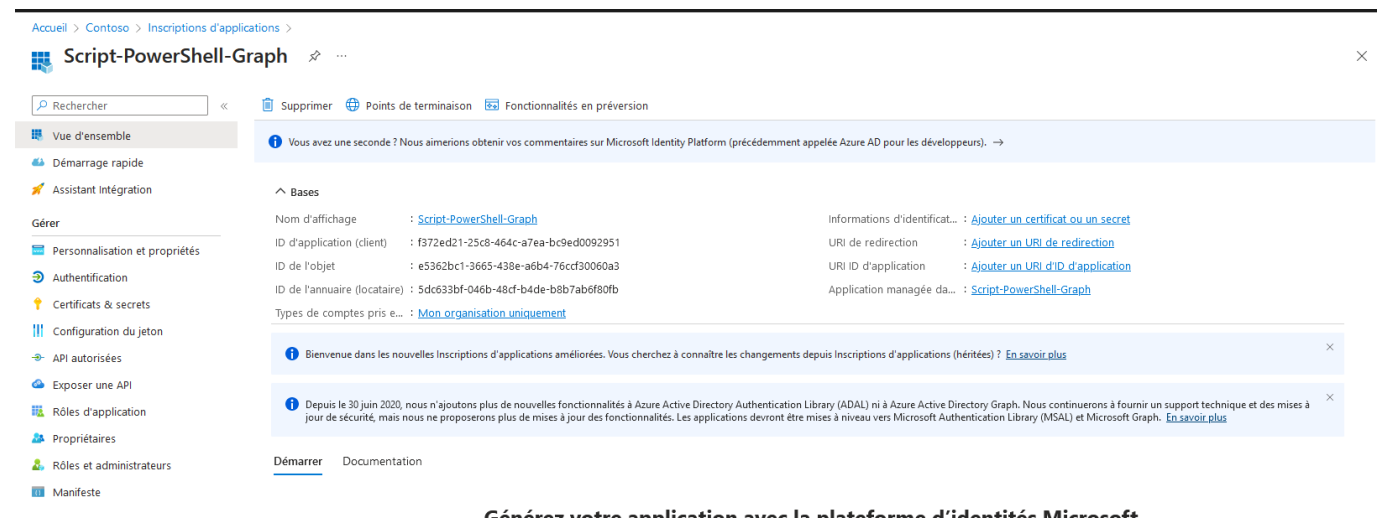
Donnez un nom à cette application, par exemple "Script-PowerShell-Graph". Conserver l'option "Comptes dans cet annuaire d'organisation uniquement" pour l'option "Types de comptes pris en charge" et pour l'URI

de redirection, laissez vide.

Cliquez sur le bouton "S'inscrire".



L'application est inscrite. Il y a deux informations qu'il faudra récupérer par la suite, car nous en aurons besoin lors de la connexion avec PowerShell : **ID d'applications (client)** et **ID de l'annuaire (locataire)**.



Désormais, nous devons accorder des autorisations à notre application

Attribuer des autorisations Microsoft Graph à l'application

Toujours sur le portail Entra ID, au sein de notre application, cliquez sur **"API autorisées"** à gauche puis au centre sur **"Ajouter une autorisation"**.

Accueil > Contoso > Inscriptions d'applications > Script-PowerShell-Graph

Script-PowerShell-Graph | API autorisées

Rechercher

Actualiser

Des commentaires ?

Vue d'ensemble

Démarrage rapide

Assistant Intégration

Gérer

Personnalisation et propriétés

Authentification

Certificats & secrets

Configuration du jeton

API autorisées

Exposer une API

Rôles d'application

Propriétaires

Rôles et administrateurs

Manifeste

Support + dépannage

Résolution des problèmes

Nouvelle demande de support

La colonne « Consentement de l'administrateur requis » indique la valeur par défaut pour une organisation. Toutefois, le consentement de l'utilisateur peut être personnalisé par autorisation, utilisateur ou application. pas refléter la valeur dans votre organisation ou dans les organisations où cette application sera utilisée. [En savoir plus](#)

Autorisations configurées

Les applications sont autorisées à appeler des API quand elles reçoivent des autorisations de la part des utilisateurs/administrateurs dans le cadre du processus de consentement. La liste des autorisations configurées doit comprendre toutes les autorisations dont l'application a besoin. [En savoir plus sur les autorisations et le consentement](#)

+ Ajouter une autorisation

✓ Accorder un consentement d'administrateur pour Contoso

API / noms des	Description	Consentement de l'...	Statut
▼ Microsoft Graph (1)			
User.Read	Délégée Activer la connexion et lire le profil utilisateur	Non	...

Pour afficher et gérer les autorisations accordées pour des applications individuelles, ainsi que les paramètres de consentement de votre client, essayez [Applications d'entreprise](#).

Nous souhaitons ajouter une autorisation **"Microsoft Graph"** et cela tombe bien c'est proposé directement.

Demander des autorisations d'API

×

Sélectionner une API

API Microsoft Graph

API utilisées par mon organisation

Mes API

API Microsoft couramment utilisées



Microsoft Graph

Tirez parti de la grande quantité de données dans Office 365, Enterprise Mobility + Security, et Windows 10. Accédez à Microsoft Entra ID, Excel, Intune, Outlook/Exchange, OneDrive, OneNote, SharePoint, Planner et à d'autres applications via un point de terminaison unique.

Cliquez sur **"Autorisations de l'application"**.

Ensuite, il faut rechercher les autorisations. Cela fonctionne sur le même principe que les étendues de la première méthode de connexion étudiée. Pour trouver l'autorisation qui permet de consulter la liste des groupes, il suffit de rechercher **"group"** et de cocher l'option **"Group.Read.All"**. Pour les utilisateurs, suivez le même principe.

Cliquez sur **"Ajouter des autorisations"** pour ajouter toutes les autorisations sélectionnées.

10 / 15

Demander des autorisations d'API

Toutes les API

Microsoft Graph

https://graph.microsoft.com/ Documents

Quel type d'autorisation votre application nécessite-t-elle ?

Autorisations déléguées

Votre application doit accéder à l'API en tant qu'utilisateur connecté.

Autorisations d'application

Votre application s'exécute en tant que service en arrière-plan ou démon sans utilisateur connecté.

Sélectionner des autorisations

développer tout

group

Autorisation	Consentement de l'administrateur...
> Calls	
▼ Group (1)	
☐ Group.Create ⓘ Create groups	Oui
☒ Group.Read.All ⓘ Read all groups	Oui
☐ Group.ReadWrite.All ⓘ Read and write all groups	Oui
> GroupMember	
> PrivilegedAccess	
> PrivilegedAssignmentSchedule	
> PrivilegedEligibilitySchedule	

Activer Windows

Il faut que l'on accorde un **consentement administration** au niveau de l'organisation pour pouvoir bénéficier de ces droits dans notre application, à savoir notre script PowerShell. Cliquez sur le bouton **"Accorder un consentement d'administrateur pour Contoso"** et validez.

Autorisations configurées

Les applications sont autorisées à appeler des API quand elles reçoivent des autorisations de la part des utilisateurs/administrateurs dans le cadre du processus de consentement. La liste des autorisations configurées doit comprendre toutes les autorisations dont l'application a besoin. [En savoir plus sur les autorisations et le consentement](#)

+ Ajouter une autorisation

✓ Accorder un consentement d'administrateur pour Contoso

API / noms des autorisations	Type	Description	Consentement de l'...	Statut
▼ Microsoft Graph (2)				...
Group.Read.All	Application	Read all groups	Oui	⚠ Pas accordé pour Conto... ...
User.Read	Déléguée	Activer la connexion et lire le profil utilisateur	Non	...

Passons à la troisième étape : la gestion du certificat.

11 / 15

Générer un certificat auto-signé avec PowerShell

Nous allons créer un **certificat auto-signé** puis l'exporter au format CER puis PFX.

Le format **PFX** est intéressant pour transférer le certificat et sa clé privée sur un autre serveur : **une étape indispensable** si vous souhaitez vous authentifier auprès de votre application depuis plusieurs serveurs différents. Le certificat (et sa clé privée) devra être déployé sur chaque machine devant se connecter à l'application.

Grâce à la commande ci-dessous, nous allons créer un certificat auto-signé et le stocker dans le magasin de certificat personnel local. Remplacez seulement "Contoso" par le nom de votre organisation.

```
$cert = New-SelfSignedCertificate -Subject "CN=Contoso" -CertStoreLocation  
"Cert:\CurrentUser\My" -KeyExportPolicy Exportable -KeySpec Signature -KeyLength  
2048 -KeyAlgorithm RSA -HashAlgorithm SHA256
```

Ensuite, il faut exporter ce certificat sur notre machine, car il va falloir le charger sur Entra ID. En PowerShell toujours, c'est faisable avec la commande "Export-Certificate". Pour ma part, je l'exporte dans "C:\TEMP" mais adaptez le chemin dans la commande ci-dessous.

```
Export-Certificate -Cert $cert -FilePath "C:\TEMP\contoso.cer"
```

```
PS C:\Users\Administrateur> Export-Certificate -Cert $cert -FilePath "C:\TEMP\contoso.cer"
```

Répertoire : C:\TEMP

Mode	LastWriteTime	Length	Name
----	-----	-----	----
-a----	27/02/2024 14:59	770	contoso.cer

Si vous avez besoin d'**utiliser ce certificat sur d'autres serveurs** (ce qui sera le cas si un autre serveur doit s'authentifier sur Microsoft Graph via PowerShell), vous devez l'**exporter au format PFX**. Si vous n'avez pas ce besoin, vous pouvez ignorer les deux commandes qui suivent.

Commençons par créer un mot de passe pour la clé privée associée à notre certificat :

```
$mdp = ConvertTo-SecureString -String "MotDePasse" -Force -AsPlainText
```

Ensuite, on **exporte au format PFX** avec `Export-PfxCertificate` en précisant le certificat via `$cert`, le mot de passe via `$mdp` et le chemin vers le **fichier de sortie au format PFX**.

```
Export-PfxCertificate -Cert $cert -FilePath "C:\temp\Contoso.pfx" -Password $mdp
```

Une fois que c'est fait, il suffira de **copier le PFX** sur les autres serveurs et de l'importer. Pensez à **stocker le mot de passe de la clé privée** dans votre gestionnaire de mots de passe préféré...

Retournez sur l'interface **Entra ID**, toujours dans notre application, et cliquez sur **"Certificats & secrets"** sur la gauche. Ensuite, cliquez sur **"Télécharger le certificat"** et chargez le fichier CER. Validez et il va apparaître dans la liste des certificats comme ceci :

Accueil > Contoso > Inscriptions d'applications > Script-PowerShell-Graph

Script-PowerShell-Graph | Certificats & secrets

Rechercher Des commentaires ?

Vue d'ensemble
Démarrage rapide
Assistant Intégration

Gérer

- Personnalisation et propriétés
- Authentification
- Certificats & secrets**
- Configuration du jeton
- API autorisées
- Exposer une API
- Rôles d'application
- Propriétaires
- Rôles et administrateurs
- Manifeste

Support + dépannage

- Résolution des problèmes
- Nouvelle demande de support

Les informations d'identification permettent aux applications confidentielles de s'identifier auprès du service d'authentification lors de la réception de jetons à un emplacement adressable web (avec un schéma HTTPS). Pour un niveau plus élevé de sécurité, nous recommandons d'utiliser un certificat (au lieu d'un secret client) comme informations d'identification.

Les certificats d'inscription d'application, les secrets et les informations d'identification fédérées se trouvent dans les onglets ci-dessous.

Certificats (0) Secrets client (0) Informations d'identification fédérées (0)

Les certificats peuvent servir de secrets pour prouver l'identité de l'application lors de la demande d'un jeton. Ils peuvent aussi être appelés des clés publiques.

Télécharger le certificat

Empreinte	Télécharger le certificat	Description	Date de début	Date d'expiration	ID de certificat
Aucun certificat n'a été ajouté pour cette application.					

Copiez la valeur **"Empreinte numérique"** (Thumbprint), car nous allons en avoir besoin pour la suite.

Connexion à Microsoft Graph avec le certificat

Voilà, la configuration est prête : nous allons pouvoir nous connecter à Microsoft Graph via PowerShell. Pour cela, il nous faut **trois informations** :

- L'ID d'applications (client) pour le paramètre **-ClientID**
- L'ID de l'annuaire (locataire) pour le paramètre **-TenantId**
- L'empreinte numérique du certificat pour le paramètre **-CertificateThumbprint**

Ce qui donne la commande **Connect-MgGraph** suivante :

```
Connect-MgGraph -ClientID f372ed21-aaaaaa-aaaa-aaaaaa-bc9ed0092951 -TenantId  
5dc633bf-aaaaaa-aaaa-aaaa-b8b7ab6f80fb -CertificateThumbprint  
5B5EB9E5xxxxxxxxxxxxxxxxxx971D273879591
```

Exécutez cette commande, et là, c'est magique on est directement authentifié ! Aucune action n'est nécessaire, c'est la combinaison de ces trois valeurs (et la présence du certificat sur la machine) qui permet de s'authentifier sur Microsoft Graph.

En termes de droits, on est limité à ce qui est déterminé au niveau des autorisations de l'application. Si l'on ajoute des droits à notre application via le portail Entra ID, il faudra se déconnecter et se reconnecter pour

que ce soit pris en compte.

```
Disconnect-MgGraph
Connect-MgGraph -ClientID f372ed21-aaaaaa-aaaa-aaaaaa-bc9ed0092951 -TenantId 5dc633bf-aaaaaaaa-aaaaa-aaaaa-b8b7ab6f80fb -CertificateThumbprint 5B5EB9E5xxxxxxxxxxxxxxxxxx971D273879591
```

Microsoft Graph : comment identifier les permissions ?

Sur le principe, Microsoft Graph est prometteur notamment sur la gestion très fine des permissions, mais cela peut rapidement devenir un casse-tête pour trouver les bonnes permissions. Je crois que chez Microsoft ils en ont conscience, car il y a un cmdlet qui permet de rechercher plus facilement des permissions : **Find-MgGraphPermission**

Si l'on souhaite obtenir toutes les permissions Microsoft Graph relatives à Microsoft Teams, on pourra exécuter la requête suivante (cela ne nécessite pas d'être authentifié auprès de Microsoft Graph).

```
PS C:\Users\Administrateur> Find-MgGraphPermission teams

PermissionType : Delegated
-----
Id              Consent Name                                     Description
-----
0e755559-83fb-4b44-91d0-4cc721b9323e User TeamsActivity.Read                               Allows the app to read your teamwork activity feed.
7ab1d787-bae7-4d5d-8db6-37ea32df9186 User TeamsActivity.Send                               Allows the app to create new activities in your teamwork activity feed, and send new activities to other users' ...
daef10fc-047a-48b0-b1a5-da4b5e72fab3 User TeamsApp.Read                                     Allows the app to read the Teams apps that are installed for you. Does not give the ability to read application...
9127ba42-f79f-43b1-be80-f23ec42377e Admin TeamsApp.Read.All                                 Allows the app to read the Teams apps that are installed for the signed-in user, and in all teams the user is a ...
2a5addc2-4d9e-4d7d-8527-5215aec410f3 User TeamsApp.ReadWrite                               Allows the app to read, install, upgrade, and uninstall Teams apps, on your behalf. Does not give the ability to...
d3f0af02-b2d2-477b-e433-14f7c3f2e1e2 Admin TeamsApp.ReadWrite.All                           Allows the app to read, install, upgrade, and uninstall Teams apps, on behalf of the signed-in user and also for...
bf3bf033-f35f-ae93-963e-47e4d874c37a User TeamsAppInstallation.ReadForChat                 Allows the app to read the Teams apps that are installed in chats that you can access. Does not give the ability...
5248dcbl-f83b-4ec3-9f4d-a4428a961a72 Admin TeamsAppInstallation.ReadForTeam                  Allows the app to read the Teams apps that are installed in teams the signed-in user can access. Does not give t...
c395395c-f79a-4dba-bc1f-8372ba9dcab4 User TeamsAppInstallation.ReadForUser                  Allows the app to read the Teams apps that are installed for you. Does not give the ability to read application...
aabb3f13-0771-4d5d-a96e-bca04c44e0df Admin TeamsAppInstallation.ReadWriteForChat              Allows the app to read, install, upgrade, and uninstall Teams apps in chats the signed-in user can access. Does ...
2e25a044-2580-450d-8859-42eeb6e996c0 Admin TeamsAppInstallation.ReadWriteForTeam              Allows the app to read, install, upgrade, and uninstall Teams apps in teams the signed-in user can access. Does ...
093f8818-d05f-49b8-95bc-9d2a73e9a43c Admin TeamsAppInstallation.ReadWriteForUser              Allows the app to read, install, upgrade, and uninstall Teams apps installed for the signed-in user. Does not gi...
0ec3576e-30a8-43b7-99e5-62f8569a4002 Admin TeamsAppInstallation.ReadWriteSelfForChat          Allows a Teams app to read, install, upgrade, and uninstall itself in chats the signed-in user can access.
0f45a9f7-6481-ae13-81bc-11b2d49f07a9 Admin TeamsAppInstallation.ReadWriteSelfForTeam          Allows a Teams app to read, install, upgrade, and uninstall itself to teams the signed-in user can access.
2070ebc1-3ce7-4922-b991-5a760c346ebc User TeamsAppInstallation.ReadWriteSelfForUser          Allows a Teams app to read, install, upgrade, and uninstall itself for you.
48638b3c-ad68-4383-8ac4-e6880ee6ca57 Admin TeamsAppInstallation.ReadWriteSelfForTeam          Read all teams' settings, on behalf of the signed-in user.
39d69650-9d2e-4223-80b6-a33539002d7e Admin TeamsAppInstallation.ReadWriteSelfForTeam          Read and change all teams' settings, on behalf of the signed-in user.
a9ff19c2-f369-4a95-9a55-ba9d460efc8e Admin TeamsAppInstallation.ReadWriteSelfForUser          Allows the app to create tabs in any team in Microsoft Teams, on behalf of the signed-in user. This does not gra...
59dacb05-e88d-4c13-a684-59f1af8cc898 Admin TeamsTab.Read.All                                 Read the names and settings of tabs inside any team in Microsoft Teams, on behalf of the signed-in user. This do...
b98bf4d1-87c6-45cc-b104-c2de4f0dafb9 Admin TeamsTab.ReadWrite.All                             Read and write tabs in any team in Microsoft Teams, on behalf of the signed-in user. This does not give access t...
ee328332-e9c2-4747-b460-f8c164b68d6e Admin TeamsTab.ReadWriteForChat                          Allows a Teams app to read, install, upgrade, and uninstall all tabs in chats the signed-in user can access.
c975d004-a06e-4fbb-9704-62daad77b949 Admin TeamsTab.ReadWriteForTeam                          Allows a Teams app to read, install, upgrade, and uninstall all tabs in teams the signed-in user can access.
c37c9b61-7762-4bff-a156-afc0005847a0 User TeamsTab.ReadWriteForUser                          Allows a Teams app to read, install, upgrade, and uninstall all tabs for you.

PermissionType : Application
-----
Id              Consent Name                                     Description
-----
70dec828-f620-4914-aa83-a29117306807 Admin TeamsActivity.Read.All                           Allows the app to read all users' teamwork activity feed, without a signed-in user.
a267235f-af13-44dc-8385-c1dc93023186 Admin TeamsActivity.Send                               Allows the app to create new notifications in users' teamwork activity feeds without a signed in user. These...
af4bd422a-4b2a-4e07-a708-8ceed48196bf Admin TeamsApp.Read.All                                 Allows the app to read the Teams apps that are installed for any user, without a signed-in user. Does not gi...
ebdb3d76-ed73-4be6-ac36-15bd04c0a535 Admin TeamsApp.ReadWrite.All                             Allows the app to read, install, upgrade, and uninstall Teams apps for any user, without a signed-in user. D...
cc7e7635-2586-41d6-adaa-88d3bcad4ee5 Admin TeamsAppInstallation.ReadForChat.All              Allows the app to read the Teams apps that are installed in any chat, without a signed-in user. Does not giv...
1f615aea-6bf9-4b05-84bd-46388e138537 Admin TeamsAppInstallation.ReadForTeam.All               Allows the app to read the Teams apps that are installed in any team, without a signed-in user. Does not giv...
9ce09811-74f7-4abd-a629-a06450422a97 Admin TeamsAppInstallation.ReadForUser.All              Allows the app to read the Teams apps that are installed for any user, without a signed-in user. Does not gi...
9e19bae1-2623-4c4f-ab6e-2664615ffba0 Admin TeamsAppInstallation.ReadWriteForChat.All          Allows the app to read, install, upgrade, and uninstall Teams apps in any chat, without a signed-in user. Do...
5dad17ba-f6cc-4954-a5a2-a0ddc95154f0 Admin TeamsAppInstallation.ReadWriteForTeam.All           Allows the app to read, install, upgrade, and uninstall Teams apps in any team, without a signed-in user. Do...
74ef0291-ca83-4d02-8c7e-d2391e64444f Admin TeamsAppInstallation.ReadWriteForUser.All          Allows the app to read, install, upgrade, and uninstall Teams apps for any user, without a signed-in user. D...
73a50599-f39c-4baf-9182-4954ac0e55cf Admin TeamsAppInstallation.ReadWriteSelfForChat.All      Allows a Teams app to read, install, upgrade, and uninstall itself for any chat, without a signed-in user.
9f67436c-5415-4e7f-8ac1-301447132630 Admin TeamsAppInstallation.ReadWriteSelfForTeam.All       Allows a Teams app to read, install, upgrade, and uninstall itself in any team, without a signed-in user.
908de74d-f8b2-4d6b-a9ed-2a17b3b78179 Admin TeamsAppInstallation.ReadWriteSelfForUser.All      Allows a Teams app to read, install, upgrade, and uninstall itself to any user, without a signed-in user.
242607bd-1d2c-432c-82eb-bdb27baa23ab Admin TeamSettings.Read.All                             Read all team's settings, without a signed-in user.
dd808a03-d9bc-4510-b7c4-cec03f3e3c8f Admin TeamSettings.ReadWrite.All                         Read and change all teams' settings, without a signed-in user.
49981c42-fd0b-4510-be05-e77b21aed95e Admin TeamsTab.Create                                    Allows the app to create tabs in any team in Microsoft Teams, without a signed-in user. This does not grant ...
46890524-499a-4bb2-ad64-1476d4f3e1cf Admin TeamsTab.Read.All                                 Read the names and settings of tabs inside any team in Microsoft Teams, without a signed-in user. This does ...
a96d855f-016b-47d7-b51c-1218a98d791c Admin TeamsTab.ReadWrite.All                             Read and write tabs in any team in Microsoft Teams, without a signed-in user. This does not give access to t...
fd9cc730-5250-40ac-b044-8dc8d20f39ea Admin TeamsTab.ReadWriteForChat.All                      Allows a Teams app to read, install, upgrade, and uninstall all tabs for any chat, without a signed-in user.
6153d4f4-f9f8-43d9-97b4-96df489ed148 Admin TeamsTab.ReadWriteForTeam.All                      Allows a Teams app to read, install, upgrade, and uninstall all tabs in any team, without a signed-in user.
425b4b59-d5af-45c8-832f-bb0b7402348a Admin TeamsTab.ReadWriteForUser.All                      Allows a Teams app to read, install, upgrade, and uninstall all tabs for any user, without a signed-in user.
```

```
Find-MgGraphPermission teams
```

Vous allez voir que cette commande retourne deux sections : **Delegated** pour l'accès délégué et **Application** pour l'accès application. En fonction de ce que vous recherchez, **précisez le type de permissions, car le nom peut varier** :

```
Find-MgGraphPermission teams -PermissionType Delegated
```

ou

```
Find-MgGraphPermission teams -PermissionType Application
```

Grâce à la sortie de cette commande et le filtre, on peut trouver plus facilement le nom des permissions. Pour les groupes, les utilisateurs, etc... On peut rechercher.

```
Find-MgGraphPermission groups  
Find-MgGraphPermission user
```

En complément, vous pouvez retrouver des informations sur cette page qui référence l'ensemble des permissions (un CTRL+F sera inévitable pour rechercher dans la page) :

- [Microsoft Docs - Référentiel des permissions](#)

Document rédigé grâce au site [Microsoft Learn](#) et [IT-Connect](#)

Axel RONDEAU - 27/02/24